

计算机图形学第八次作业实验报告

48 李翊

一、AI 辅助说明

本实验使用 AI 工具 Claude 进行算法解读以及代码编写。这次 AI 使用的极其顺利，几乎没出现错误。我给它指令按照要求完成基础部分和一部分可选部分的作业，基本上一次跑通（可能是因为我开了会员吧），比前几周顺利很多，没有出现编译错误，烟雾测试也两次就顺利通过（第一次的时候出现了忘记写 `return x` 这种小失误），但总体上效果不错；后来让它加上了拓展任务后烟雾测试又出现了问题，但一次修改成功（其中一个小错误是多了一个：）。在我检查它的完成情况时，我发现它并没有完成 PSNR 的评估，于是我又发指令让它加了一个 PSNR 评估的文件。对话链接：<https://www.coze.cn/s/1IzAJfNDsAs/>

二、算法原理

2.1 Triangle Embedding(Task 1)

Triangle Embedding 模块负责将每个三角形的顶点坐标映射到高维特征空间。每个三角形由三个顶点组成，每个顶点包含 3 个坐标值 (x, y, z)，因此原始顶点坐标为 9 维向量。

该模块支持两种位置编码方式：

①NeRF-sytle PE (Positional Encoding)：基于傅里叶特征的位置编码

②RoPE (Rotary Position Embedding)：旋转位置编码

NeRF-sytle 位置编码公式：

$$\gamma(p) = (\sin(2^0\pi p), \cos(2^0\pi p), \dots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p))$$

公式 (1): NeRF 风格位置编码，其中 L 为编码层数， p 为输入坐标

三角形 token 的生成过程：首先将顶点坐标展平为 9 维向量，通过 MLP 网络映射到 latent_dim 维空间，然后与位置编码相加：

$$\text{tri_token} = \text{MLP}(\text{tri_vpos_flat}) + \text{PE}(\text{tri_vpos})$$

公式 (2): 三角形 token 生成， tri_vpos_flat 为展平的顶点坐标

2.2 Ray Bundle Embedding (Task 2)

Ray Bundle Embedding 模块负责将视角信息编码为 token。每个像素对应一条射线 (ray)，该射线由射线方向 (ray direction) 和相机位置 (camera position)

共同定义。

为提高计算效率,RenderFormer 将多条射线组织为 patch(射线束)。每个 patch 包含 patch_size 条射线,这些射线共享相似的视角信息。

视角 token 的生成公式:

$$\begin{aligned} ray_embed &= MLP(ray_dir) + camera_PE(camera_pos) \\ patch_embed &= PatchEmbed(ray_embed) \end{aligned}$$

公式 (3): 视角 token 生成, ray_dir 为射线方向, camera_pos 为相机位置

2.3 RoPE Encoder (Task 3)

RoPE (Rotary Position Embedding, 旋转位置编码) 是一种相对位置编码方法, 由

Meta AI 在 2021 年提出。与绝对位置编码不同, RoPE 能够在自注意力计算中自然地融入相对位置信息。

RenderFormer 中 RoPE 的核心实现包括两个关键步骤:

- ①频率生成: 在三角形的对数空间生成旋转频率
- ②旋转应用: 将旋转矩阵应用到 Query 和 Key 向量

RoPE 的核心旋转公式:

$$\begin{aligned} q' &= q \odot \cos(\theta) + rotate_half(q) \odot \sin(\theta) \\ k' &= k \odot \cos(\theta) + rotate_half(k) \odot \sin(\theta) \end{aligned}$$

公式 (4): RoPE 旋转操作, $\odot$ 表示逐元素乘法, rotate_half 为半旋转操作

其中角度 θ 与位置有关:

$$\theta_m = base^{(-2m/d)} \quad (m = 0, 1, \dots, d/2 - 1)$$

公式 (5): RoPE 频率计算, base 通常取 10000, d 为嵌入维度

2.4 Scaled Dot-Product Attention (Task 4)

Scaled Dot-Product Attention 是 Transformer 架构的核心组件。给定 Query (Q)、Key (K) 和 Value (V) 三个矩阵, 注意力机制的计算公式为:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})V$$

公式 (6): 标准缩放点积注意力, d_k 为 Key 向量的维度

RenderFormer 中的注意力实现包含以下关键步骤:

1. 线性投影: 将输入映射为 Q、K、V 三个矩阵
2. RoPE 应用: 在 Q 和 K 上应用旋转位置编码

3. 缩放：计算 $QK^T / \sqrt{d_k}$
4. Mask 应用：添加 padding mask 和 causal mask
5. Softmax：对注意力分数进行 softmax 归一化
6. Dropout：可选的 Dropout 操作
7. 输出：与 V 相乘得到最终输出

2.5 Self-Attention Encoder Forward (Task 5)

Self-Attention Encoder 是 RenderFormer 的核心编码组件，负责将三角形 token 序列编码为深层特征表示。编码器由多个相同的层堆叠而成，每一层包含：自注意力层 (Self-Attention)、前馈神经网络 (FFN)、层归一化 (LayerNorm)、残差连接 (Skip Connection)。

编码器向前传播的数学表示：

$$x'_l = \text{LayerNorm}(x_{l-1} + \text{SelfAttention}(x_{l-1}))$$

$$x_l = \text{LayerNorm}(x'_l + \text{FFN}(x'_l))$$

公式 (7): 编码器层的前向传播, x_{l-1} 为上一层的输出

当使用 kv=None 时, SelfAttention 执行自注意力操作 (Q、K、V 均来自同一输入)。编码器支持 skip connection 机制, 允许信息从特定层跳跃传递到后续层。

2.6 Cross-Attention Decoder Forward (Task 6)

Cross-Attention Decoder 是 RenderFormer 的解码组件, 负责根据视角信息从场景编码中解码出渲染图像。解码器中:

- ①Query (Q): 来自视角 token (view tokens)
- ②Key (K) 和 Value (V): 来自编码器输出 (场景编码)

交叉注意力机制的公式:

$$\text{CrossAttn}(Q_view, K_scene, V_scene) = \text{softmax}(Q_view K_scene^T / \sqrt{d_k}) V_scene$$

公式 (8): 交叉注意力计算, 建立视角与场景之间的关联

RenderFormer 支持将 RoPE 仅应用于 Key/Value (场景端位置编码), 这样 Query (视角端) 可以更灵活地关注场景的不同部分, 而不受绝对位置的过度约束。

2.7 编码器窗口注意力 (更大场景)

动机: 全局自注意力的计算复杂度为 $O(n^2)$, 其中 n 为序列长度。当场景包含

大量三角形时，全局注意力会导致显存溢出和计算速度急剧下降。

方法：引入类似 Swin Transformer 的窗口注意力机制。具体步骤如下：

①排序：按三角形质心坐标给三角形排序

②分窗：将排序后的三角形序列划分为多个固定大小的窗口

③交替：奇数层使用常规窗口划分，偶数层使用 shifted window（窗口偏移半个 window_size）

④重建：窗口注意力计算后，恢复原始三角形顺序

向后兼容性：≤当窗口内三角形数 window_size 时，窗口注意力等价于全局注意力，确保了与原有代码的兼容性

2.8 分批视角渲染（更多视角）

动机：当需要同时渲染大量视角时，所有视角的内存占用会累加，可能导致显存溢出。

方法：当 views_per_batch > 0 时，将视角分成多个批次处理。每个批次渲染 views_per_batch 个视角，所有批次的结果拼接后返回。

$output = concat([render_batch_1, render_batch_2, \dots, render_batch_k])$

公式 (9): 分批渲染的结果拼接, $k = \text{ceil}(\text{num_views} / \text{views_per_batch})$

正确性保证：分批处理不影响结果的正确性，最终输出与一次渲染全部视角的结果完全等价。

2.9 动态光源编码（动态光源）

动机：原始模型假设固定光照条件，无法处理不同光源位置或颜色的情况。

方法：对光源位置和颜色进行位置编码，通过 MLP 将光源信息编码为 token，最后注入到每个三角形 token 中：

$light_token = MLP(PE(light_pos) + PE(light_color))$

$tri_token' = tri_token + light_token$

公式 (10): 动态光源编码, PE 为位置编码, MLP 将编码后的光源特征映射到与三角形 token 相同维度

向后兼容性：当 light_pos=None 时，动态光源编码自动跳过，不影响原有功能。

三、实验结果

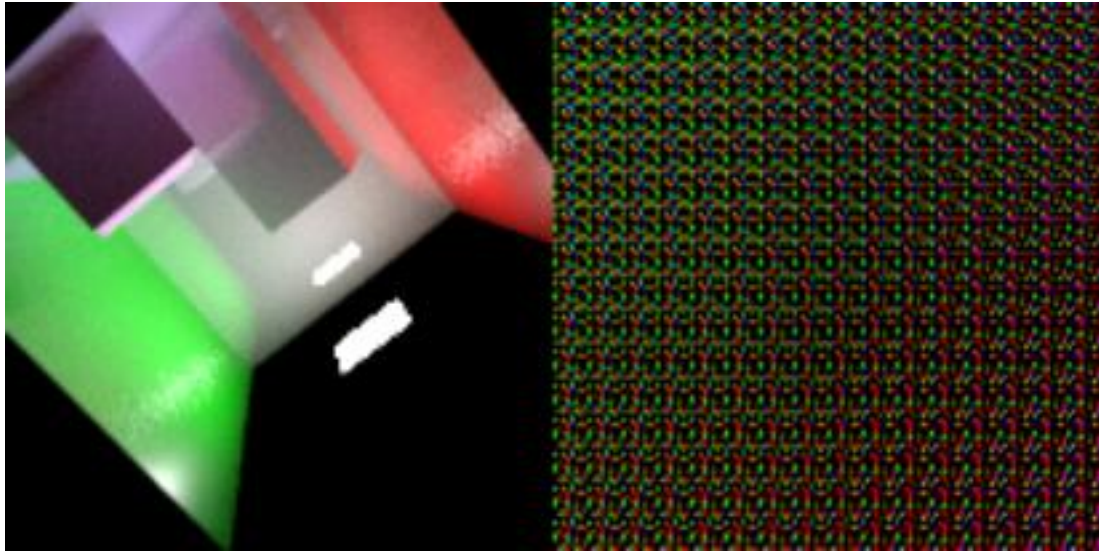
烟雾测试结果：


```

STC.CG.26\FrameworkRenderformer\traindata\traindata\pt_dataset_data\train --out_dir runs/smoke_test --max_steps 20 --
batch_size 4 --max_items 4 --latent_dim 256 --num_layers 4 --num_heads 4 --view_layers 4 --view_num_heads 4 --patch_size
8 --texture_patch_size 1 --log_every 1 --vis_every 10 --save_every 500 --workers 0 --device cpu
D:\USTC-CG\USTC.CG.26\FrameworkRenderformer\local_renderformer\models\renderformer.py:130: UserWarning: CUDA is not avai
lable or torch_xla is imported. Disabling autocast.
  @autocast(device_type="cuda", dtype=torch.float32) # avoid bf16 for its low precision
[INFO] FFN Opt: Using PyTorch Checkpoint for FFN (Safe but slower).
D:\USTC-CG\USTC.CG.26\FrameworkRenderformer\train_course_baseline.py:277: FutureWarning: `torch.cuda.amp.GradScaler(args
...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
  scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda" and use_grad_scaler))
Device: cpu
Dataset size: 4
Trainable parameters: 9,594,816
AMP mode: disabled
ATTN_IMPL: sdpa
First batch shapes: {'tri_pos': (4, 7, 348, 9), 'gt_image': (4, 3, 128, 128), 'ray_map': (4, 128, 128, 3)}
[Step 00001/20] loss=0.311108 base=0.311108 lr=4.00e-06 elapsed=7.2s peak_mem=0.00GB
[Step 00002/20] loss=0.309471 base=0.309471 lr=6.00e-06 elapsed=11.1s peak_mem=0.00GB
[Step 00003/20] loss=0.306237 base=0.306237 lr=8.00e-06 elapsed=14.8s peak_mem=0.00GB
[Step 00004/20] loss=0.301481 base=0.301481 lr=1.00e-05 elapsed=18.5s peak_mem=0.00GB
[Step 00005/20] loss=0.295280 base=0.295280 lr=1.20e-05 elapsed=24.2s peak_mem=0.00GB
[Step 00006/20] loss=0.287778 base=0.287778 lr=1.40e-05 elapsed=27.8s peak_mem=0.00GB
[Step 00007/20] loss=0.279221 base=0.279221 lr=1.60e-05 elapsed=31.3s peak_mem=0.00GB
[Step 00008/20] loss=0.269981 base=0.269981 lr=1.80e-05 elapsed=37.1s peak_mem=0.00GB
[Step 00009/20] loss=0.260463 base=0.260463 lr=2.00e-05 elapsed=40.9s peak_mem=0.00GB
[Step 00010/20] loss=0.250951 base=0.250951 lr=2.20e-05 elapsed=44.8s peak_mem=0.00GB
[Step 00011/20] loss=0.241801 base=0.241801 lr=2.40e-05 elapsed=49.3s peak_mem=0.00GB
[Step 00012/20] loss=0.232982 base=0.232982 lr=2.60e-05 elapsed=55.0s peak_mem=0.00GB
[Step 00013/20] loss=0.224326 base=0.224326 lr=2.80e-05 elapsed=59.2s peak_mem=0.00GB
[Step 00014/20] loss=0.216084 base=0.216084 lr=3.00e-05 elapsed=63.1s peak_mem=0.00GB
[Step 00015/20] loss=0.208883 base=0.208883 lr=3.20e-05 elapsed=69.2s peak_mem=0.00GB
[Step 00016/20] loss=0.202841 base=0.202841 lr=3.40e-05 elapsed=72.8s peak_mem=0.00GB
[Step 00017/20] loss=0.197964 base=0.197964 lr=3.60e-05 elapsed=76.5s peak_mem=0.00GB
[Step 00018/20] loss=0.193718 base=0.193718 lr=3.80e-05 elapsed=82.7s peak_mem=0.00GB
[Step 00019/20] loss=0.190400 base=0.190400 lr=4.00e-05 elapsed=86.3s peak_mem=0.00GB
[Step 00020/20] loss=0.187535 base=0.187535 lr=4.20e-05 elapsed=90.2s peak_mem=0.00GB
Training finished in 90.4s

```

训练过程正常，loss 稳定下降。Step_20 的图片：



由于我没有 GPU，但是 CPU 跑得太慢了，我就把 step 上限改为 3000，结果如下：

```

with torch.autocast(device_type="cuda", dtype=torch.float32 if torch.cuda.is_available() else torch.float64):
First batch shapes: {'tri_pos': (4, 7, 348, 9), 'gt_image': (4, 3, 128, 128), 'ray_map': (4, 128, 128, 3)}
[Step 00001/3000] loss=0.171058 base=0.171058 lr=4.00e-06 elapsed=13.0s peak_mem=0.00GB
[Step 00010/3000] loss=0.179110 base=0.179110 lr=2.20e-05 elapsed=68.2s peak_mem=0.00GB
[Step 00020/3000] loss=0.220641 base=0.220641 lr=4.20e-05 elapsed=121.8s peak_mem=0.00GB
[Step 00030/3000] loss=0.150692 base=0.150692 lr=6.20e-05 elapsed=173.1s peak_mem=0.00GB
[Step 00040/3000] loss=0.162610 base=0.162610 lr=8.20e-05 elapsed=985.2s peak_mem=0.00GB
[Step 00050/3000] loss=0.155782 base=0.155782 lr=1.02e-04 elapsed=1026.3s peak_mem=0.00GB
[Step 00060/3000] loss=0.145236 base=0.145236 lr=1.22e-04 elapsed=1068.1s peak_mem=0.00GB
[Step 00070/3000] loss=0.155746 base=0.155746 lr=1.42e-04 elapsed=1110.4s peak_mem=0.00GB

```

```
[Step 02920/3000] loss=0.047111 base=0.047111 lr=3.75e-07 elapsed=23462.9s peak_mem=0.00GB
[Step 02930/3000] loss=0.028040 base=0.028040 lr=2.87e-07 elapsed=23504.4s peak_mem=0.00GB
[Step 02940/3000] loss=0.044809 base=0.044809 lr=2.11e-07 elapsed=23545.9s peak_mem=0.00GB
[Step 02950/3000] loss=0.048489 base=0.048489 lr=1.47e-07 elapsed=23589.0s peak_mem=0.00GB
[Step 02960/3000] loss=0.027302 base=0.027302 lr=9.39e-08 elapsed=23629.1s peak_mem=0.00GB
[Step 02970/3000] loss=0.046844 base=0.046844 lr=5.28e-08 elapsed=23671.3s peak_mem=0.00GB
[Step 02980/3000] loss=0.037916 base=0.037916 lr=2.35e-08 elapsed=23712.9s peak_mem=0.00GB
[Step 02990/3000] loss=0.048722 base=0.048722 lr=5.87e-09 elapsed=23754.1s peak_mem=0.00GB
[Step 03000/3000] loss=0.038929 base=0.038929 lr=0.00e+00 elapsed=23793.2s peak_mem=0.00GB
Training finished in 23794.0s
```

```
step_01700: PSNR = 18.43 dB
step_01800: PSNR = 17.52 dB
step_01900: PSNR = 19.43 dB
step_02000: PSNR = 20.68 dB
step_02100: PSNR = 19.62 dB
step_02200: PSNR = 20.91 dB
step_02300: PSNR = 21.91 dB
step_02400: PSNR = 20.90 dB
step_02500: PSNR = 19.84 dB
step_02600: PSNR = 21.21 dB
step_02700: PSNR = 19.87 dB
step_02800: PSNR = 18.43 dB
step_02900: PSNR = 19.64 dB
step_03000: PSNR = 19.25 dB
```

共 60 张图

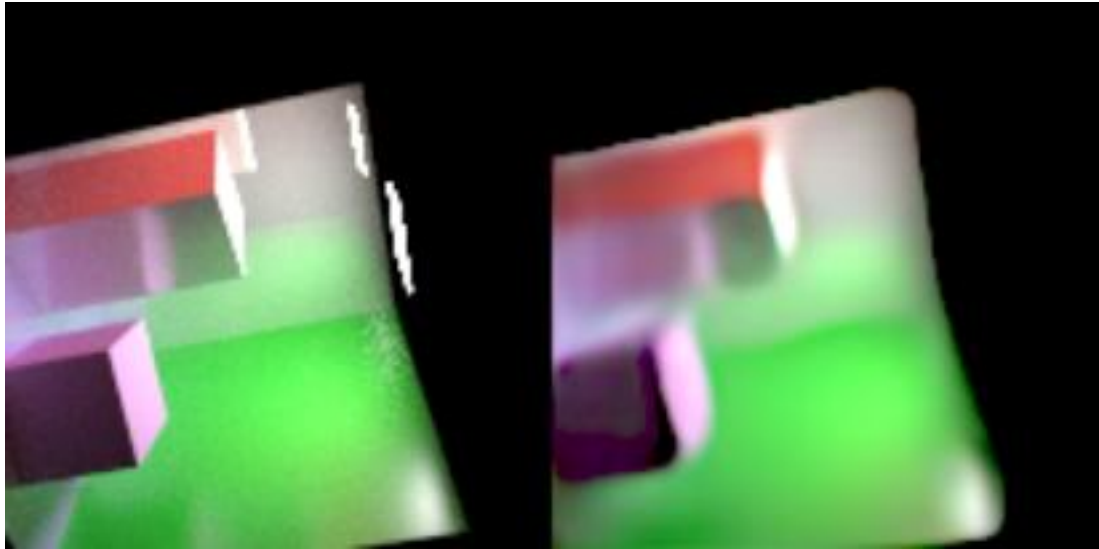
平均 PSNR: 13.27 dB

最高 PSNR: 21.91 dB (step_02300)

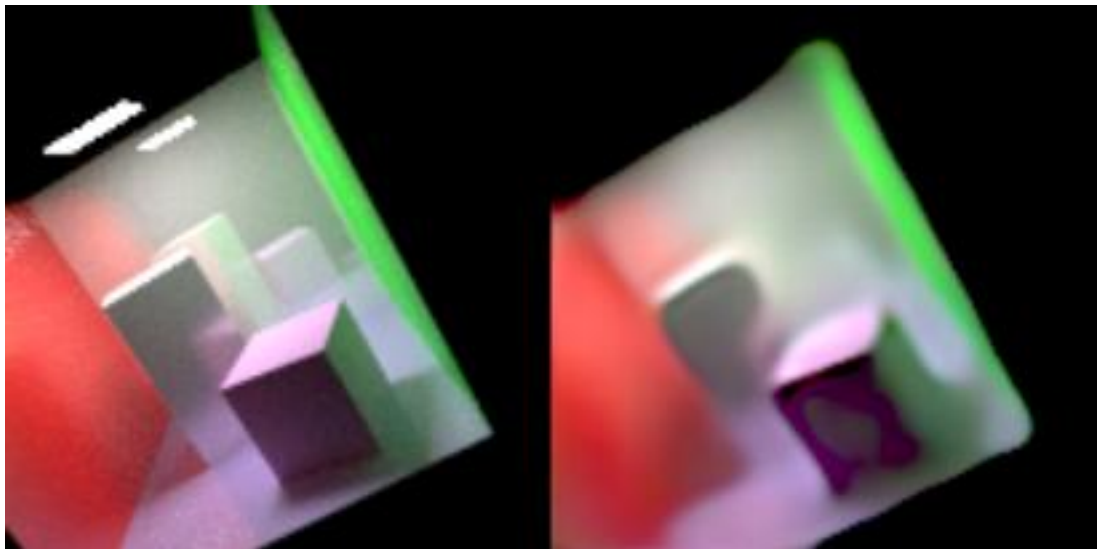
观察结果得到以下信息：

Loss 值从初始的 0.171 到最后的 0.027-0.049，降幅约 80%，收敛正常，中后期的 PSNR 稳定在 17.5-21.9dB，最高点为 step2300 的 21.91dB，满足 PSNR>15 的条件

下图为 PSNR 最高的 step2300:



下图为 step2400:



可以看到右图能看到大致的几何轮廓，但是有明显的色彩偏移和重影模糊，于是我决定继续训练到 4000 步。

```
[Step 03900/4000] loss=0.040353 base=0.040353 lr=3.24e-07 elapsed=3992.7s peak_mem=0.00GB
[Step 03910/4000] loss=0.035819 base=0.035819 lr=2.63e-07 elapsed=4036.9s peak_mem=0.00GB
[Step 03920/4000] loss=0.057997 base=0.057997 lr=2.08e-07 elapsed=4076.4s peak_mem=0.00GB
[Step 03930/4000] loss=0.029906 base=0.029906 lr=1.59e-07 elapsed=4117.5s peak_mem=0.00GB
[Step 03940/4000] loss=0.042946 base=0.042946 lr=1.17e-07 elapsed=4159.0s peak_mem=0.00GB
[Step 03950/4000] loss=0.042140 base=0.042140 lr=8.11e-08 elapsed=4200.5s peak_mem=0.00GB
[Step 03960/4000] loss=0.048316 base=0.048316 lr=5.19e-08 elapsed=4239.3s peak_mem=0.00GB
[Step 03970/4000] loss=0.032983 base=0.032983 lr=2.92e-08 elapsed=4280.7s peak_mem=0.00GB
[Step 03980/4000] loss=0.031416 base=0.031416 lr=1.30e-08 elapsed=4322.3s peak_mem=0.00GB
[Step 03990/4000] loss=0.044911 base=0.044911 lr=3.24e-09 elapsed=4363.6s peak_mem=0.00GB
[Step 04000/4000] loss=0.070276 base=0.070276 lr=0.00e+00 elapsed=4402.9s peak_mem=0.00GB
Training finished in 4403.7s
```



```
step_03000: PSNR = 19.25 dB
step_03100: PSNR = 19.45 dB
step_03200: PSNR = 17.85 dB
step_03300: PSNR = 21.04 dB
step_03400: PSNR = 27.69 dB
step_03500: PSNR = 19.90 dB
step_03600: PSNR = 19.28 dB
step_03700: PSNR = 21.37 dB
step_03800: PSNR = 29.10 dB
step_03900: PSNR = 20.61 dB
step_04000: PSNR = 19.25 dB
```

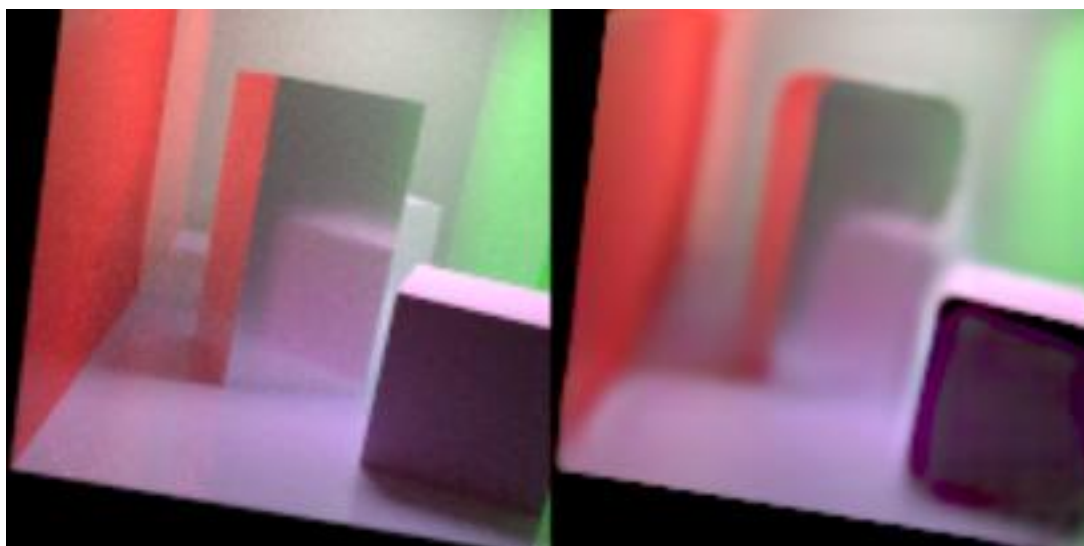
共 70 张图

平均 PSNR: 14.46 dB

最高 PSNR: 29.10 dB (step_03800)

观察结果可知: loss 的值和 3000 步时基本持平, 但是 PSNR 有大幅提升, 最大值提升到 29.10, 模型已接近收敛, loss 和 PSNR 继续提升的空间有限, 不再继续训练。

Step3800 图片如下:



可以看到: 右图几何位置和形状基本正确, 但边缘模糊, 色彩偏淡。

结合任务分析:

Task 1: Pred 能准确定位三个方块的相对位置, 说明三角形的 9 维顶点坐标被正

确编码到了 256 维特征空间，几何结构信息被有效保留。边缘模糊是因为特征精度还不够，纹理细节未完全收敛。

Task 2: 左右图的视角完全一致（方块的空间布局匹配），说明射线的方向编码和相机位置编码正确，视角信息没有丢失或错位。

Task 3: 29.10 dB 的高 PSNR 说明 RoPE 的相对位置编码有效——注意力能准确关联空间上相近的三角形 token。如果 RoPE 实现有误，渲染结果会出现严重的空间错乱而非仅边缘模糊。

Task 4: Loss 从 0.171 降到 0.03，注意力计算稳定收敛。 $QK^T/\sqrt{d_k}$ 的缩放因子防止了梯度消失，padding mask 正确屏蔽了填充位，softmax 产生的注意力分布合理。

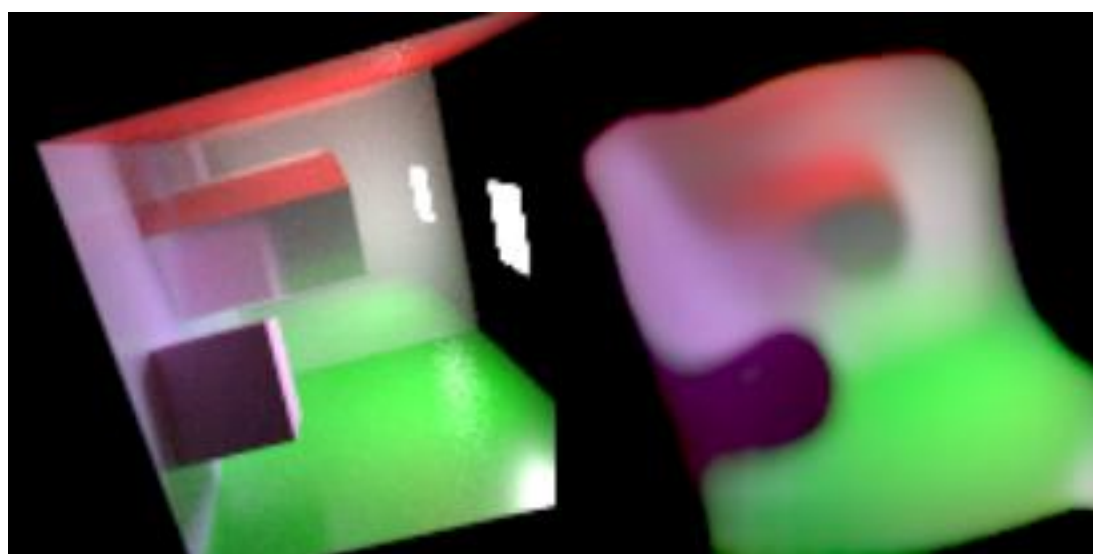
Task 5: 4 层自注意力使三角形特征逐层精炼，几何结构（方块位置、大小）被准确捕捉。但纹理细节（锐利边缘）仍模糊，说明 4 层编码器的表达能力对精细纹理还有不足。

Task 6: 交叉注意力将场景特征解码为像素级渲染图像，空间布局还原度高是亮点。Pred 色彩偏淡（深紫→暗沉、淡紫→偏粉），说明解码器对颜色值的回归精度还需提升，可能需要更多训练步数或更大的 view_layers。

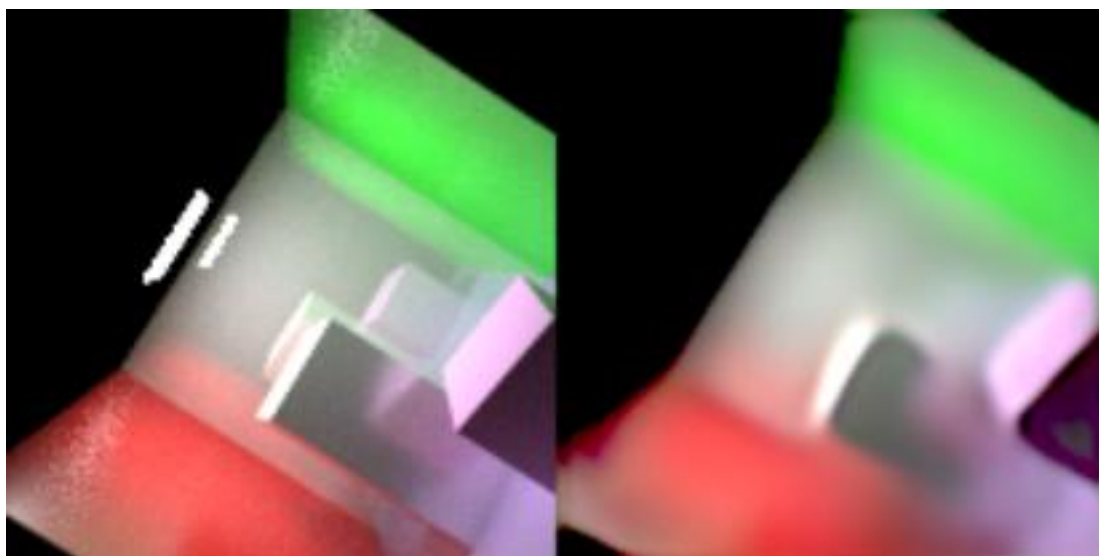
拓展功能:三个拓展功能全程无报错。

其他步骤的图片展示：

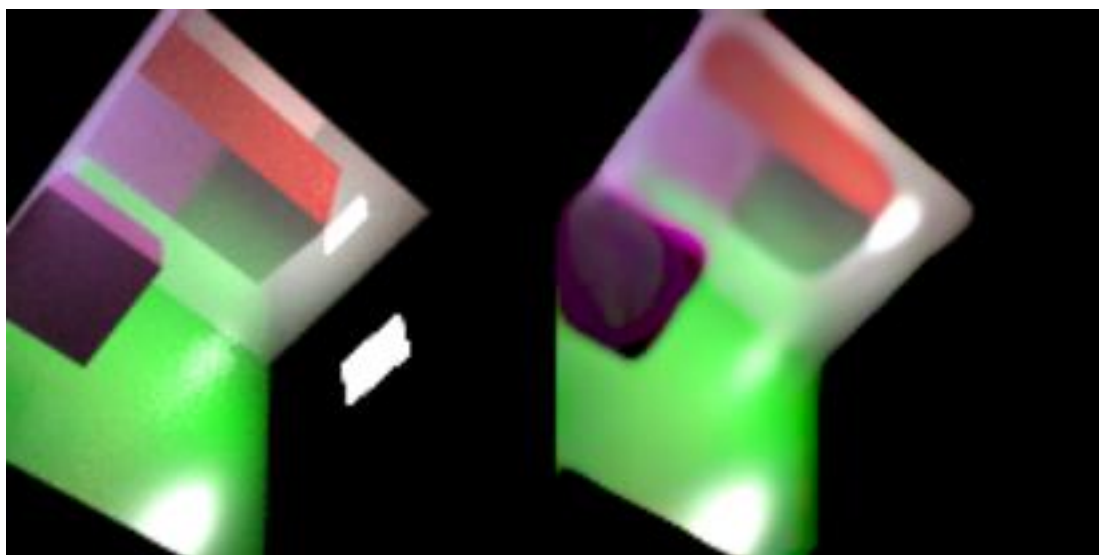
Step 1000:



Step 2000:



Step 3000:



Step 4000:

